

02. Uso de LLMs para generar código

Ciencia de datos para la toma de decisiones II

**Jorge
Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Anuncios



Hice un repositorio del curso en Github, al cual iré cargando material: <https://github.com/JuveCampos/TC2002B.651-Ciencia-de-datos>

TC2002B.651-Ciencia-de-datos Public

main 1 Branch 0 Tags Go to file Add file Code

JuveCampos [20250819] Modificacion Readme y carga de presentaciones 9d79075 · 7 minutes ago 3 Commits

00_Presentaciones	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago
01_Ejercicios prácticos/20250819_sesion_02	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago
98_archivos_curso	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago
99_assets	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago
.gitignore	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago
CLAUDE.md	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago
README.html	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago
README.md	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago
TC2002B.651-Ciencia-de-datos.Rproj	[20250819] Modificacion Readme y carga de presentacio...	7 minutes ago

README

TC2002B.651-Ciencia-de-datos



Repositorio de archivos del curso de Ciencia de datos para la toma de decisiones impartido en el Tec de Monterrey en el semestre de Otoño-Invierno del 2025.

****Profesores:** ** M.C. Jorge Juvenal Campos Ferreira y Jose Manuel Toral.

****Objetivo del curso:** ** Diseñe, implemente y comunique soluciones de ciencia de datos de forma integral, combinando el uso de programación en R, análisis estadístico, IA generativa, procesamiento de texto, análisis

Anuncios

- **Igualmente, los archivos para los ejercicios prácticos los iré cargando a la siguiente carpeta:**
https://drive.google.com/drive/folders/1--76NtTS7MLXM-azOZl4ecOIbBMtWoHW?usp=share_link
- **También los compartiré por correo, si se registran en esta hoja de google sheets:** <https://docs.google.com/spreadsheets/d/1ohfNz61cGEp78trgWK5jjW6s18YjqMFCrTi2kLBOrqk/edit?gid=0#gid=0>

Programa de la clase

- Ejercicios para realizar un prompt
- Introducción a **Claude Code**
- Introducción super básica a la terminal
- Ejercicio guiado de generación de código
- Introducción a RStudio
- Proyectos en R

Fundamentos de ingeniería de prompts



Prompt: Es la instrucción o pregunta que le damos al modelo para que genere una respuesta.

Ingredientes mínimos de un buen prompt

Enfoque: La actividad que queremos que realice el modelo.

Contexto: La información previa que queremos que el modelo tome en consideración al momento de procesar el prompt

Límites: Lo que el modelo debe o no debe incluir en su respuesta

Rol: El rol que debe asumir el modelo al momento de dar la respuesta.

Ejemplos de Prompts

Prompt sencillo

Realiza una investigación completa sobre el INFONAVIT de México. Incluye su historia, funcionamiento actual, noticias recientes y cualquier dataset disponible público. Proporciona información detallada y actualizada.

¿Cuál dará el mejor resultado?

Prompt más elaborado

```
<rol>
Actúa como un investigador especializado en instituciones de vivienda de México con experiencia en análisis de políticas públicas y
manejo de datos gubernamentales.
</rol>

<enfoque>
Realizar una investigación exhaustiva y estructurada sobre el Instituto del Fondo Nacional de la Vivienda para los Trabajadores
(INFONAVIT), abarcando múltiples dimensiones: histórica, institucional, operativa y de disponibilidad de datos.
</enfoque>

<contexto>
El INFONAVIT es una institución tripartita mexicana creada en 1972 que administra el fondo de vivienda de los trabajadores formales. Es
crucial para el sector habitacional mexicano y maneja grandes volúmenes de datos sobre créditos, viviendas y trabajadores cotizantes. La
investigación debe considerar su evolución, reformas recientes, impacto socioeconómico y transparencia en datos.
</contexto>

<tareas_especificas>
1. Investigar la historia institucional del INFONAVIT desde 1972 hasta la actualidad
2. Analizar noticias y desarrollos recientes (últimos 2 años)
3. Identificar y catalogar todos los datasets públicos disponibles
4. Examinar programas actuales y productos crediticios
5. Evaluar el marco regulatorio y reformas recientes
</tareas_especificas>

<límites>
- NO incluir información de otras instituciones de vivienda sin relación directa con INFONAVIT
- NO especular sobre datos no públicos o confidenciales
- NO incluir opiniones políticas partidistas
- Sí incluir únicamente fuentes verificables y oficiales
- Sí proporcionar enlaces directos a datasets cuando sea posible
- Sí mantener un enfoque analítico y objetivo
</límites>

<formato_salida>
Estructura la investigación en las siguientes secciones:
1. Resumen ejecutivo
2. Historia institucional
3. Marco legal y regulatorio actual
4. Programas y productos actuales
5. Noticias y desarrollos recientes (2023-2025)
6. Datasets disponibles (con URLs directas)
7. Análisis de transparencia y acceso a información
8. Conclusiones y recomendaciones para investigadores
</formato_salida>

<fuentes_prioritarias>
- Sitio oficial de INFONAVIT
- Portal de datos abiertos del gobierno mexicano
- INAI (Instituto Nacional de Transparencia)
- Informes anuales oficiales
- Boletines de prensa institucionales
- Bases de datos del IMSS relacionadas
</fuentes_prioritarias>
```

Ventana de contexto

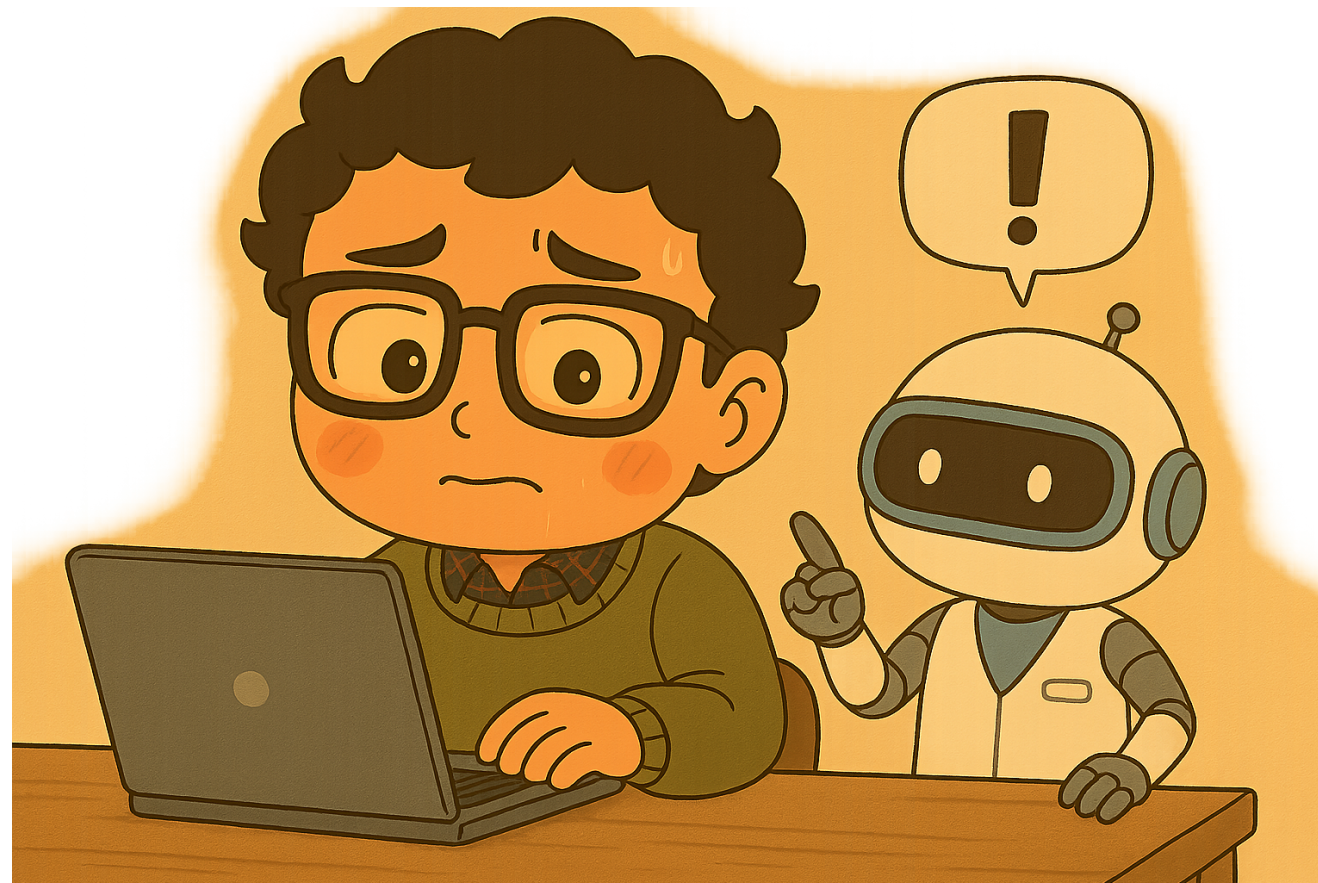
★ La **ventana de contexto** (o "longitud de contexto") de un **modelo de lenguaje grande (LLM)** es la cantidad de texto, en **tokens**, que el modelo puede considerar o "recordar" en cualquier momento.

Una ventana de contexto más grande permite que un **modelo de IA** procese entradas más largas e incorpore una mayor cantidad de información en cada salida.

Meta prompting

El **metaprompting** es una técnica avanzada de ingeniería de prompts que utiliza un modelo de lenguaje grande (LLM) para generar, optimizar o refinar prompts para otros LLMs.

En esencia, se trata de **usar la IA para ayudar a crear prompts** más efectivos, lo que permite obtener mejores resultados del modelo.



Ejercicio

Redacte un *prompt* para investigar un tema que le sea de interés o para generar un texto que le sea de interés.

Redáctelo tomando en cuenta **Enfoque, Contexto, Límites y Rol**.

Utilice la sintaxis XML del ejemplo anterior (*envolver cada elemento en los símbolos ">" y "<"*). ¿Considera que redactar este prompt le representa alguna ventaja?

Una vez que lo termine, pídele al LLM que tome el rol de un "ingeniero de prompt" y que te mejore el prompt que acabas de escribir.

LLMs para generar código

Modelos de generación de código

Los LLMs pueden escribir, corregir y optimizar código porque fueron entrenados no solo con lenguaje natural, sino también con **repositorios de código** y documentación técnica.

Aplicaciones:

- **Autocompletar** y escribir funciones enteras.
- Detectar y corregir errores (*debugging*).
- Explicar código existente.
- Convertir descripciones en código (prompt → script).
- Traducir entre lenguajes de programación.

Modelos de generación de código

- Hay muchas herramientas para generar código con IA. ChatGPT, Claude, Gemini o Grok funcionan bien
- También hay herramientas especializadas para mezclar IA y código, como Cursor, o CLIs (*Command Line Interface*), como Claude Code o Gemini CLI.
- Las herramientas especializadas en código facilitan la programación, y en algunos casos ejecutan y prueban el código.



Codex

OpenAI Codex CLI – Getting Started

Updated: hace 3 días

Overview

OpenAI Codex CLI is an open-source command-line tool that brings the power of our latest reasoning models directly to your terminal. It acts as a lightweight coding agent that can **read, modify, and run code on your local machine** to help you build features faster, squash bugs, and understand unfamiliar code. Because the CLI runs locally, your source code never leaves your environment unless you choose to share it.

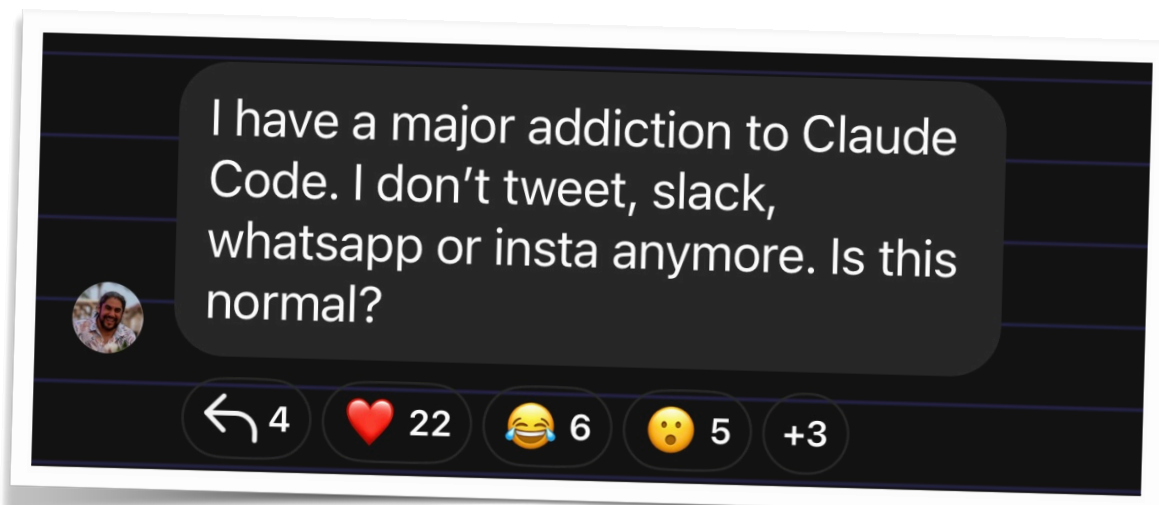


Los desarrollos se están generando muy rápido!!

For more details about Codex, please check out the Github repo directly - <https://github.com/openai/codex>.

Modelos de generación de código

- Mi herramienta favorita (ahorita a agosto del 2025) es Claude Code.
- Es un CLI que permite acceder a carpetas de trabajo, leer y escribir archivos y también generar código.
- Claude Code ha tenido un buen desempeño al momento de trabajar con código de R, y particularmente en generar aplicaciones shiny básicas.



Claude Code

- Instrucciones de instalación: https://www-anthropic-com.translate.google/claude-code?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

Se necesita usar terminal.

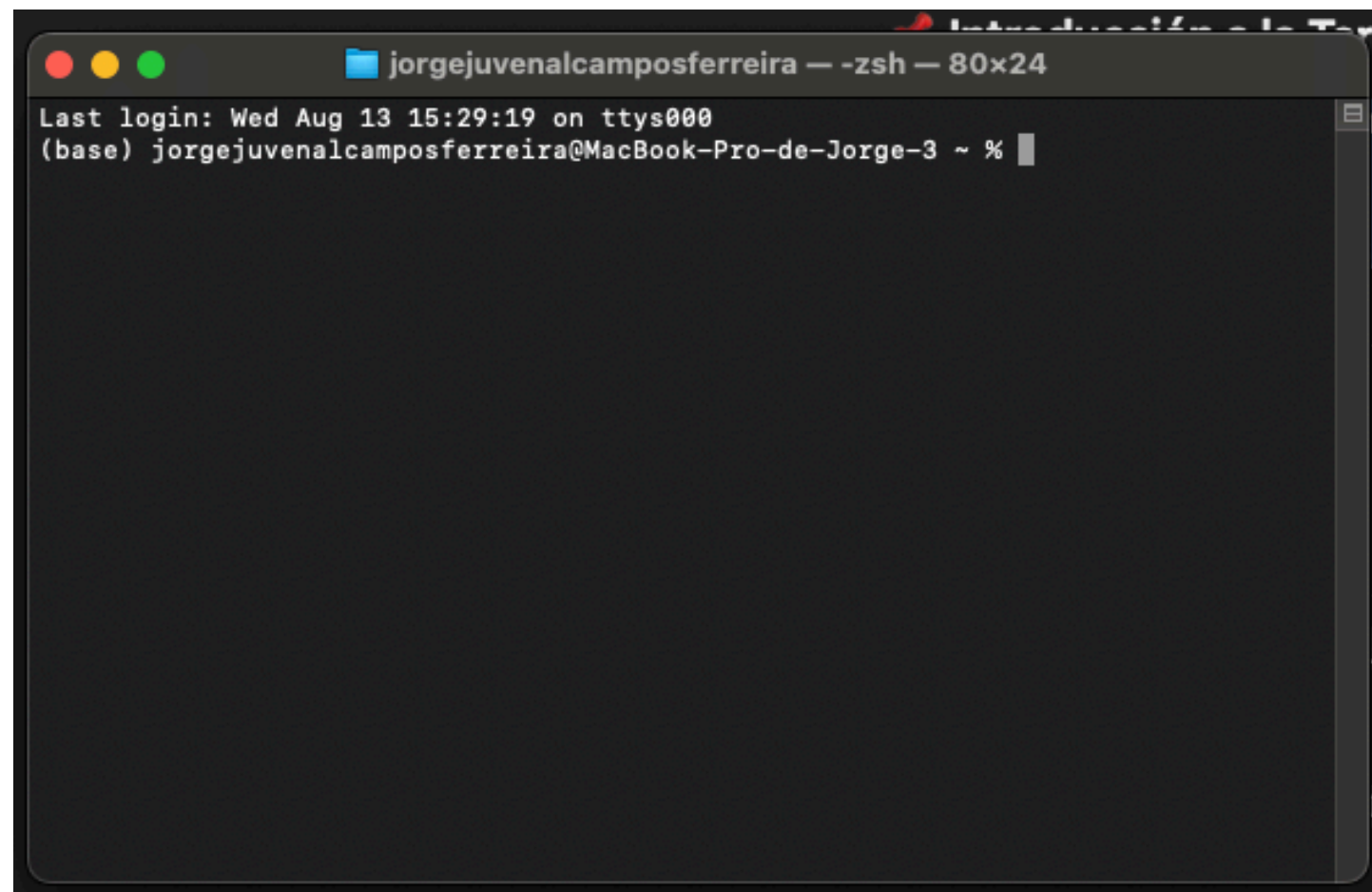
Antes de la instalación, hay que instalar Node.js 18+



Terminal

La **terminal** (o línea de comandos) es una herramienta para **interactuar directamente con el sistema operativo** mediante texto, sin usar ventanas o íconos.

Permite navegar por carpetas, ejecutar programas y administrar archivos con comandos.



Ejercicio guiado

1. Instale alguna herramienta CLI en su computadora.
2. Ejecutelo y corra un prompt que genere un archivo para generar una gráfica básica en ggplot.
3. Ejecute el código y verifique que funcione.

Vibe Coding



Es una **técnica de programación** dependiente de IA en la que una persona describe un problema en unas pocas oraciones como instrucciones para un LLM.

El LLM genera software, cambiando el rol del programador de la codificación manual a la guía, prueba y refinamiento del código fuente generado por IA

Vibe Coding



El **vibe coding** es un término acuñado por Andrej Karpathy (cofundador de OpenAI y ex-líder de IA en Tesla) en febrero de 2025. Como él mismo lo describió: "**There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists**"



Vibe Coding

MY CODE DOESN'T WORK

I HAVE NO IDEA WHY

MY CODE WORKS

I HAVE NO IDEA WHY

VIBE CODING

EXPECTATIONS

This is amazing!
I will change
the world!!

REALITY

NOOO... WHY
DID YOU DO THAT?
YOU BROKE
EVERYTHING!!!

BUILD AN APP.
I'LL VIBE-CHECK
IT LATER.

VIBE CODING ISN'T
GOING TO REPLACE ME

BECAUSE MY FUTURE
IS FIXING VIBE CODING

VIBE
CODING

VIBE
DEBUGGING

VIBE
CODING

Sugerencia: Instalar Github Desktop



Y hacer una cuenta de Github

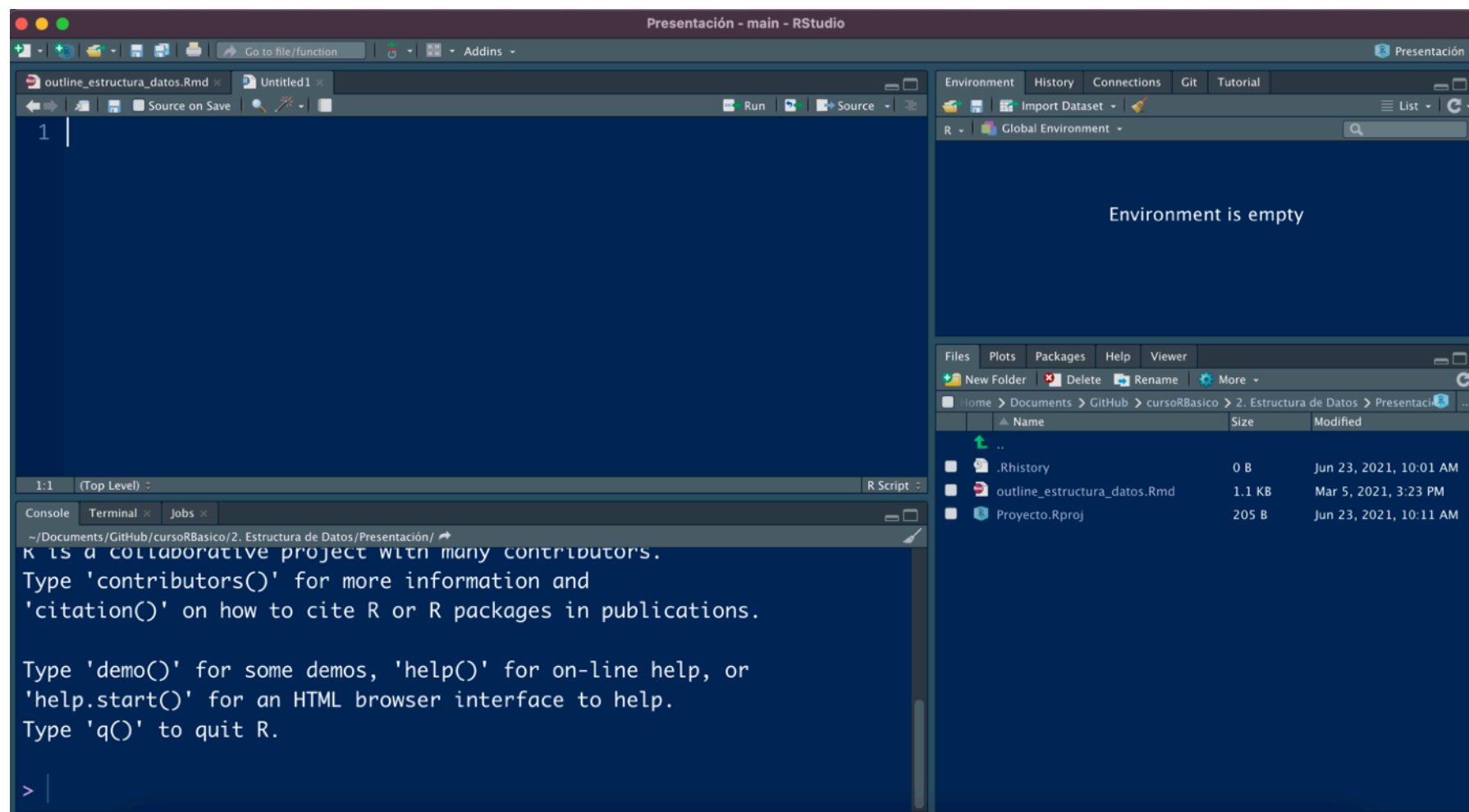
02. Introducción a RStudio

Ciencia de datos para la toma de decisiones II

Jorge
Juvenal
Campos Ferreira

✉ juvenal.campos@tec.mx

RStudio es un programa que provee un entorno de desarrollo (IDE) que nos da las herramientas necesarias para poder programar en R.



Ventanas



The screenshot shows the RStudio interface with the following panels highlighted:

- Editor de texto (Yellow):** Contains R code for creating a data structure.
- Ambiente (Pink):** Shows the Environment pane with a table of values.
- Consola (Green):** Shows the R console output for the executed code.
- Visualizador (Red):** Shows the Files pane with a list of files in the project directory.

Code in Editor de texto:

```
1 # Librerías ----
2 library(tidyverse)
3
4 # Bases de datos ----
5 personas <- c("Juvenal", "María")
6 años <- c(29, 30)
7 escuelas <- c("Colmex", "UNAM")
8 sexo <- factor(c("M", "F"),
9               levels = c("M", "F"))
10 sabe_r <- c(TRUE, FALSE)
```

Output in Consola:

```
+               levels = c("M", "F"))
> sexo
[1] M F
Levels: M F
> sabe_r <- c(TRUE, FALSE)
```

Table in Ambiente:

Values	
años	num [1:2] 29...
pers...	chr [1:2] "J...
sabe...	logi [1:2] T...

Files in Visualizador:

Name	Size
..	
.Rhistory	0
outline_estructura_datos.Rmd	1
Proyecto.Rproj	2

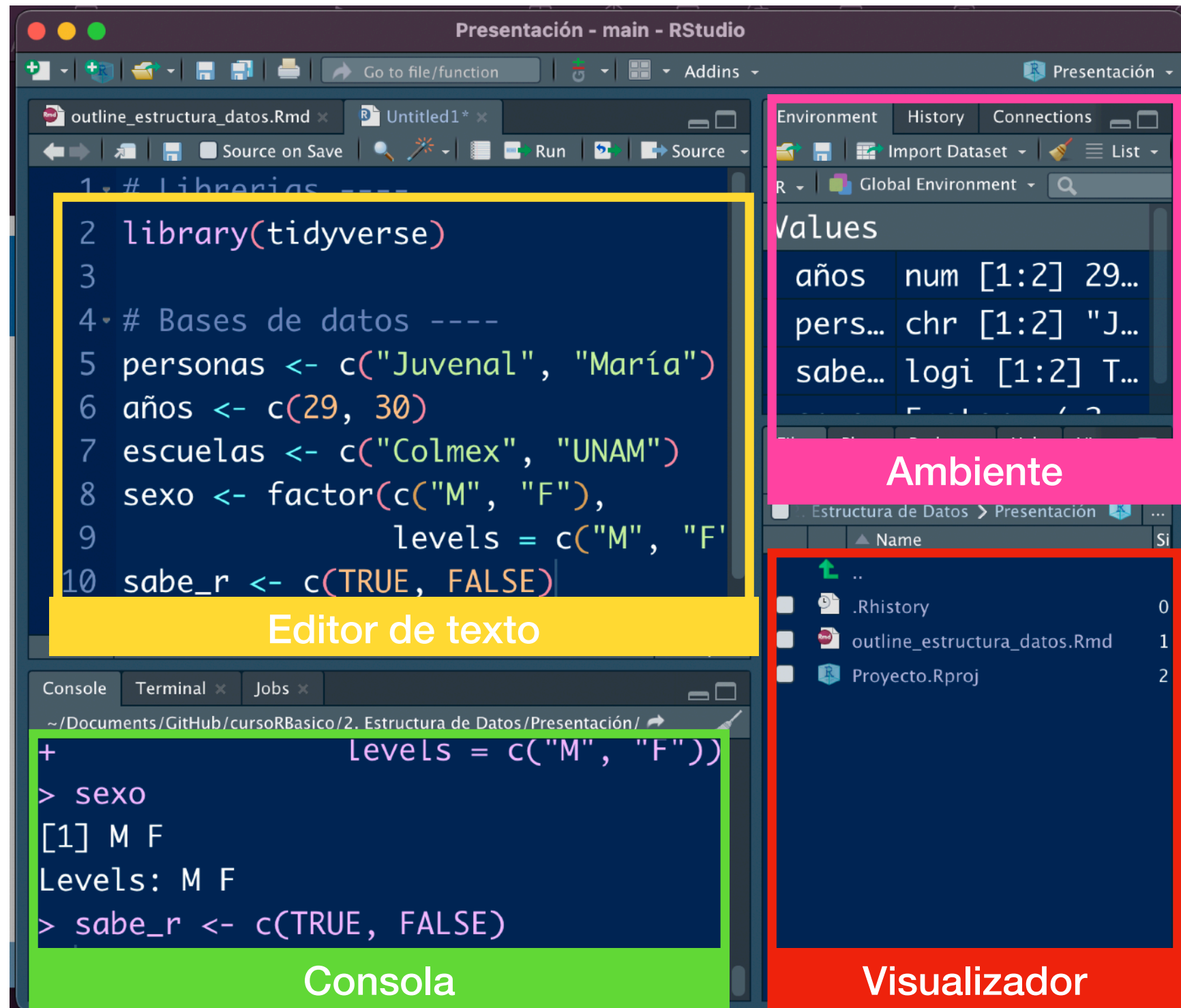
Ventanas



Editor de texto

Sección del programa en la cual registramos las instrucciones que se van a correr en R. Estas instrucciones se guardan en scripts para volver a ellos más adelante.

Acá se pueden escribir códigos de R, HTML, Python, CSS, Markdown, etc.



Ventanas



Consola

Sección en la cual se ejecuta el código que vamos a escribir en el editor de texto.

Igualmente, podemos correr acá código de R que no requerimos guardar para más adelante.

The screenshot shows the RStudio interface with four windows highlighted by colored boxes and labels:

- Editor de texto** (Yellow box): The source editor showing R code for creating a dataset structure.
- Ambiente** (Pink box): The Environment pane showing the current environment with variables like 'años', 'pers...', and 'sabe...'.
- Consola** (Green box): The Console pane showing the execution of the R code, including the output for the 'sexo' variable.
- Visualizador** (Red box): The Viewer pane showing the file explorer with files like '.Rhistory', 'outline_estructura_datos.Rmd', and 'Proyecto.Rproj'.

The code in the Editor de texto window is as follows:

```
1. # Librerías ----
2 library(tidyverse)
3
4 # Bases de datos ----
5 personas <- c("Juvenal", "María")
6 años <- c(29, 30)
7 escuelas <- c("Colmex", "UNAM")
8 sexo <- factor(c("M", "F"),
9               levels = c("M", "F"))
10 sabe_r <- c(TRUE, FALSE)
```

The output in the Consola window is as follows:

```
+               levels = c("M", "F"))
> sexo
[1] M F
Levels: M F
> sabe_r <- c(TRUE, FALSE)
```

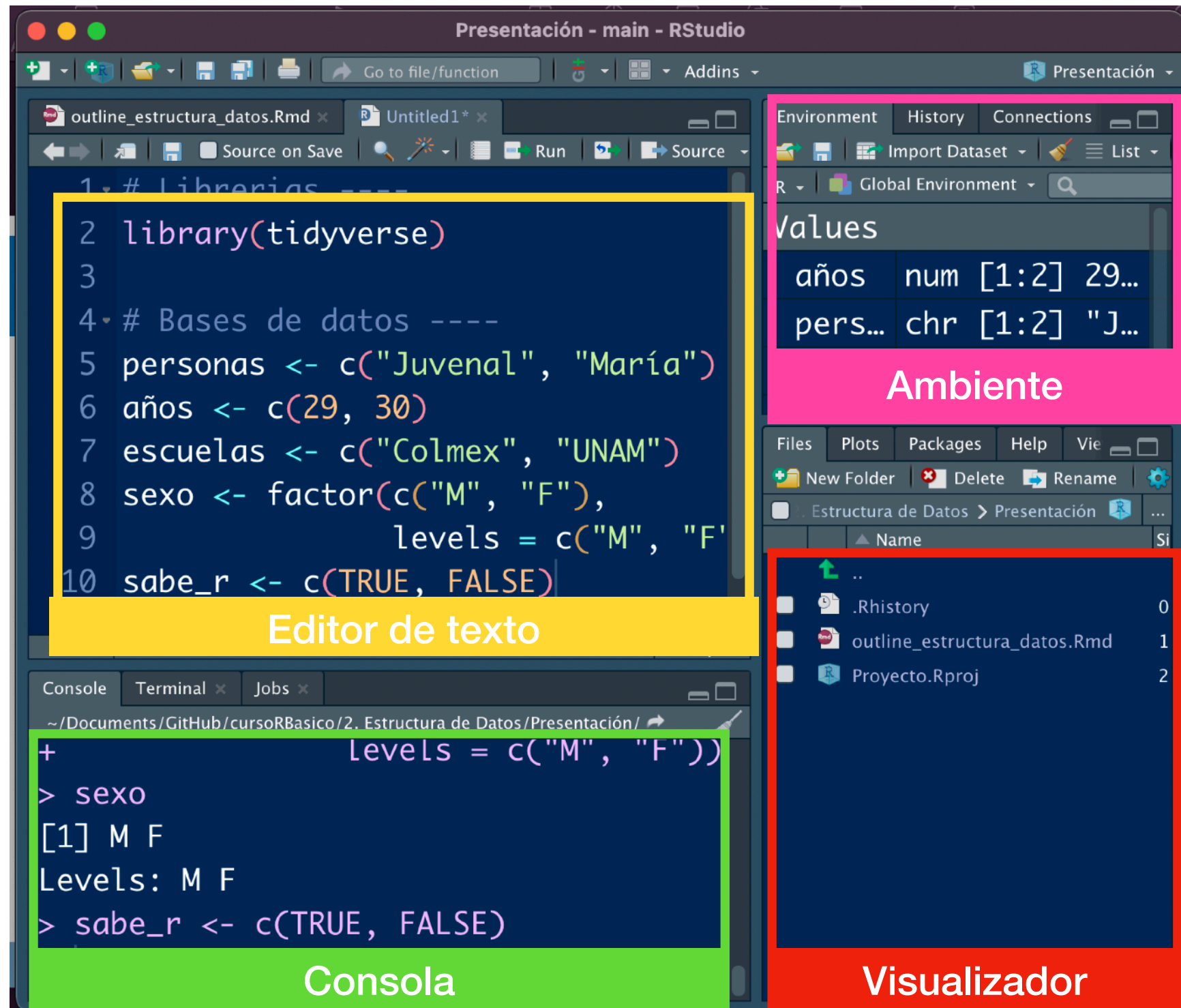
Ventanas



Visualizador

Sección para visualizar cosas:

- 1) Archivos ubicados en nuestro directorio de trabajo
- 2) Gráficas estáticas generadas con ggplot2 o RBase.
- 3) Las librerías instaladas en nuestro RStudio.
- 4) Las visualizaciones web generadas con R.



Personalización

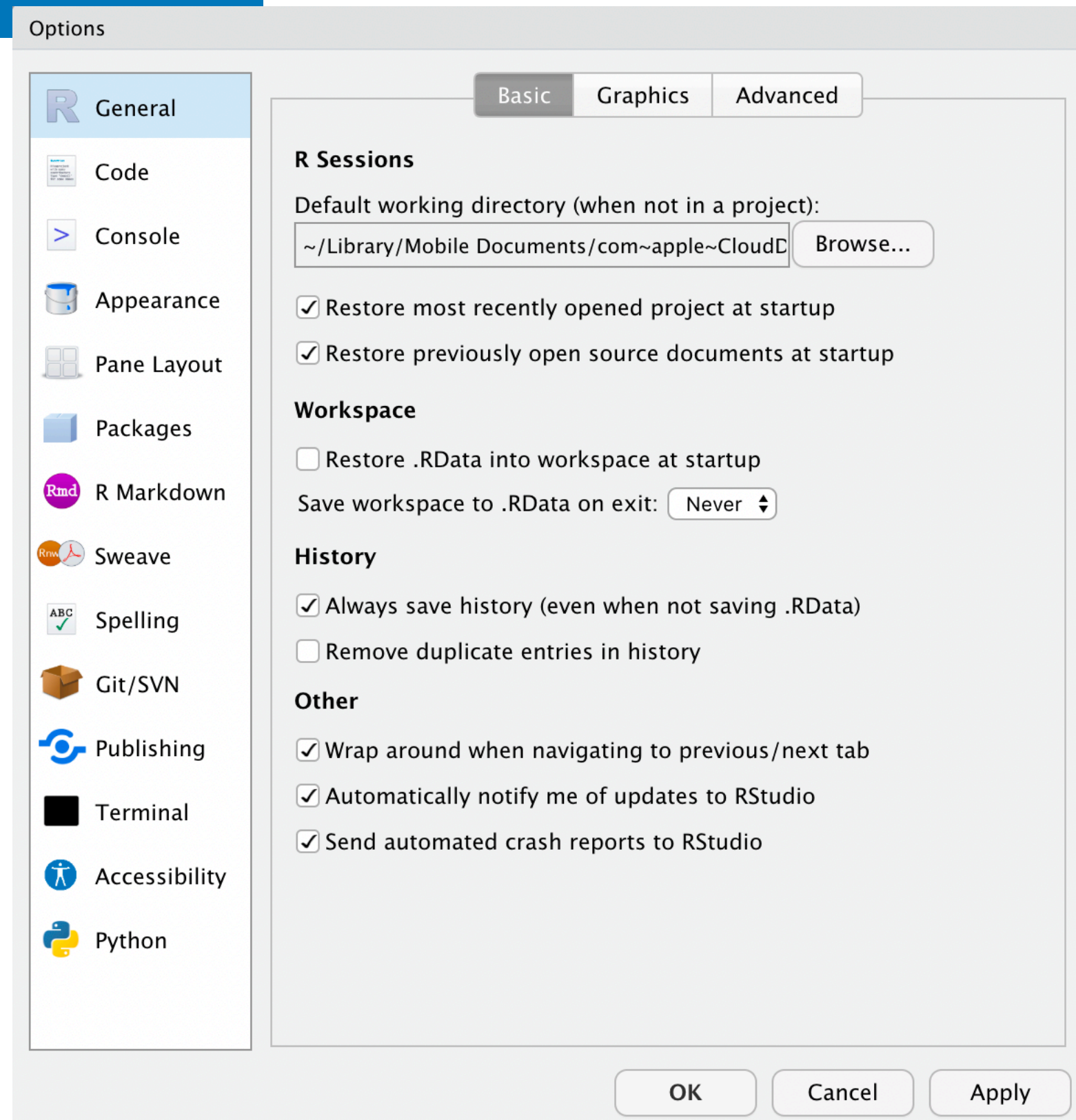


Para personalizar RStudio
vamos a

Tools > Global Options

Podemos configurar:

- como se visualiza el código,
- los colores de las ventanas,
- el tamaño y fuente de las letras,
- el espacio a ocupar del código,
- las cuentas para publicar resultados, etc.





Archivos nativos de R

Al trabajar con R y RStudio, generamos cuatro tipos de archivos nativos de R, estos son los siguientes:

- *.Rproj,
- *.RData,
- *.R y
- *.rds.

Los scripts son aquellos archivos que terminan en *.r